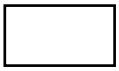
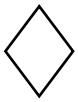
12. Entity Relationship Diagram

This section covers the theoretical description of the Entity-Relationship design of the Medical Appointment & Prescription System, including a description of the notations used, database entities, attributes, and cardinalities.

12.1 ER Components & Symbols

Symbol / Element Name	Visual Representation	Description
Entity		Represents a core database table or real-world object within the system(e.g., USERS, PATIENTS, DOCTORS).
Relationship		Represents the operational association or interaction linking two entities together(e.g., Has Profile, Books, Attends).
Attribute		Represents the specific properties, data fields, or columns belonging to an entity(e.g., username, age, specialization).
Primary key		Identifies the unique attribute used to uniquely distinguish each record within an entity table(e.g., user_id, patient_id).
One to one	1:1	Defines a strict mapping constraint where a single record in one entity relates to exactly one record in another(e.g., Users to Patients).
One to many	1:N	Defines a mapping constraint where a single record in an entity relates to multiple records in another(e.g., Patients to Appointments).

12.2 Entity and Attributes

i. Users

- **Purpose:** Stores the data related to central authentication such as usernames, the secure hashed passwords, and roles of all the users of the system.
- **Attributes**
  - user_id (Primary Key, INT)
  - username (VARCHAR)
  - password_hash (VARCHAR)
  - role (VARCHAR)

ii. Patients

- **Purpose:** Stores the contact and personal information of all the people eligible to receive services through the system.
- **Attributes:**

  - patient_id (Primary Key, INT)
  - full_name (VARCHAR)
  - age (INT)
  - gender (VARCHAR)
  - contact_number (VARCHAR)
  - residential_address (TEXT)
  - blood_group (VARCHAR)
  - registration_date (DATETIME)

iii. Doctors

- **Purpose:** Stores the personal and professional information of all the medical personnel such as their specializations, contact information, and schedule.
- **Attributes:**

  - doctor_id (Primary Key, INT)
  - full_name (VARCHAR)
  - specialization (VARCHAR)
  - contact_number (VARCHAR)
  - working_hours (VARCHAR)
  - consultation_fee(DECIMAL)
  - availability_status (BOOLEAN)

iv. Appointments

- **Purpose:** Stores the schedule of doctors' appointments including the date and time of each appointment
- **Attributes:**

  - appointment_id (Primary Key, INT)
  - appointment_date (DATE)
  - appointment_time (TIME)
  - status (VARCHAR)
  - created_at (DATETIME)

v. Prescriptions

- **Purpose**: Stores the medical treatment records of all the patients in electronic form including the prescribed drugs and notes made by doctors during each visit.
- **Attributes:**

  - prescription_id (Primary Key, INT)
  - medicines_and_dosage (TEXT)
  - clinical_notes (TEXT)
  - prescription_date (DATETIME)

vi. Medical_histories

- **Purpose:** Captures detailed clinical notes, symptoms, diagnoses, and treatment progress during physicians' consultations with patients
- **Attributes:**

- history_id (Primary Key, INT)
- recorded_at (DATETIME)
- symptoms (TEXT)
- diagnosis (TEXT)
- treatment_notes (TEXT)

vii. Medicine_reminders

- **Purpose**: Manages dosage alarms and tracking of medication to ensure patients that they take the dose of medication according to the prescribed schedule.
- **Attributes :**

  - reminder_id (Primary Key, INT)
  - medicine_name (VARCHAR)
  - dosage (VARCHAR)
  - scheduled_at (DATETIM|E)
  - snoozed_until (DATETIME)
  - status (VARCHAR)

12.3 Relationship

Entity one	Relation	Entity two	Relation Linkage
Users	Register	Patients	The Patients table has a foreign key (registered_by) that points to the primary key (user_id) of the Users table, indicating who among the staff or admin created a patient record..
Users	has profile	Doctors	The Doctors table contains a foreign key (user_id) linked to the primary key (user_id) in the Users table.
Patients	Books	Appointments	The Appointments table contains a foreign key (patient_id) linked to the primary key (patient_id) in the Patients table
Doctors	Attends	Appointments	.The Appointments table contains a foreign key (doctor_id) linked to the primary key (doctor_id) in the Doctors table.
Appointments	Generates	Prescriptions	The Prescriptions table contains a foreign key (Appointment_id) linked to the primary key (appointment_id) in the Appointments
Patients	Receives	Prescriptions	The Prescriptions table contains a foreign key (patient_id) linked to the

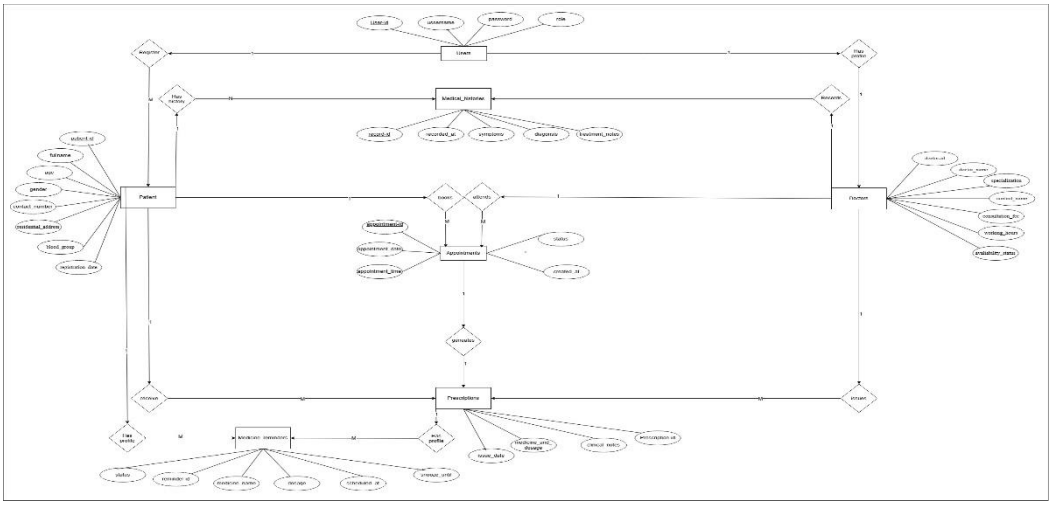
			primary key (patient_id) in the Patients table.
Doctors	Issues	Prescriptions	The Prescriptions table contains a foreign key (doctor_id) linked to the primary key (doctor_id) in the Doctors table.
Patients	Has History	Medical_histories	The medical_histories table contains a foreign key (patient_id) that links to the patients.patient_id, establishing a relationship where a patient accumulates multiple historical clinical records.
Doctors	Records	Medical_histories	The medical_histories table contains a foreign key (doctor_id) that links to the doctors.doctor_id. This denotes which medical practitioner diagnosed and authored the clinical notes.
Patients	Receieves	Medicine_reminders	The medicine_reminders table contains a foreign key (patient_id) that links to the patients.patient_id. This maintains a relationship for tracking personal medication alerts.
Prescriptions	Triggers	Medicine_reminders	The medicine_reminders table contains a foreign key (prescription_id) that links to the prescriptions.prescription_id. This establishes a relationship where a single prescription generates specific scheduled dosage alerts.

12.4 Cardinality Mapping

Entity One	**Cardinality**	**Entity Two**	**Mechanism**
Users	1:N	Patients	A single staff or administrative user can register multiple patient profiles over time (registered_by foreign key linkage)
Users	1:1	Doctors	Each professional doctor’s data profile is tied to a singular

			administrative user's account login.
Patients	1:N	Appointments	Every single registered patient is allowed and able to make multiple appointment slots for consultations.
Doctors	1:N	Appointments	A single medical doctor arranges and schedules multiple appointment slots for consultations within their working hours.
Appointments	1:1	Prescriptions	Every finished appointment slot for consultation generates a singular digital prescription record.
Patients	1:N	Prescriptions	A single patient holds multiple historical prescription records, relating to their medical history.
Doctors	1:N	Prescriptions	A medical doctor issues multiple prescription records to multiple patients, within their working hours.
Patients	1:N	Medical_histories	A single patient record contains many historical clinical records over several patient visits connected by patient_id
Doctors	1:N	Medical_histories	An attending doctor creates and maintains many medical history records for different patients linked by doctor_id.
Patients	1:N	Medicine_reminders	A single patient record deals with many dosage alerts for ongoing treatments connected through patient_id.
Prescriptions	1:N	Medicine_reminders	One individual prescription slip creates many scheduled medication reminders connected using prescription_id.

12.5 Diagram


User
Patient
Doctor
Receptionist
Medical_history
Appointments
Prescriptions
Lab_tests
Invoices
Pharmacy
Notifications
Audit_logs
Departments
Rooms
Beds
Admissions
Insurance
Emergency_contacts
Medical_images
Prescription_items
Lab_test_results
Payments
Refunds
Shifts
Leaves
Feedback
Complaints
Vaccinations
Allergies
Surgeries
Family_history
Vitals
Referrals
Follow_ups
Consent_forms
Ambulance_requests
Blood_bank
Donors
Inventory
Maintenance
Assets
Visitors
Parking
Security_logs
Training
Certifications
Performance_reviews
Salaries
Benefits
Expenses
Budgets
Vendors
Contracts
Meetings
Minutes
Projects
Tasks
Documents
Archives
Backups
Logs
Settings
Roles
Permissions
Sessions
Tokens
Keys
Certificates
Licenses
Subscriptions
Plans
Coupons
Discounts
Taxes
Currencies
Exchange_rates
Languages
Translations
Templates
Emails
SMS
Notifications
Alerts
Reminders
Schedules
Events
Holidays
Weekends
Shifts
Rosters
Timesheets
Overtime
Leaves
Attendance
Payroll
Bonuses
Deductions
Loans
Advances
Insurance
Claims
Refunds
Audits
Logs
Backups
Restores
Migrations
Seeds
Factories
Tests
Coverage
Reports
Dashboards
Widgets
Charts
Graphs
Exports
Imports
Integrations
Webhooks
APIs
Endpoints
Requests
Responses
Errors
Exceptions
Middleware
Guards
Policies
Gates
Scopes
Observers
Events
Listeners
Jobs
Queues
Workers
Schedules
Commands
Tasks
Processes
Threads
Locks
Semaphores
Mutexes
Transactions
Savepoints
Rollbacks
Commits
Connections
Pools
Drivers
Adapters
Repositories
Services
Controllers
Routes
Middleware
Requests
Responses
Errors
Exceptions
Logs
Backups
Restores
Migrations
Seeds
Factories
Tests
Coverage
Reports
Dashboards
Widgets
Charts
Graphs
Exports
Imports
Integrations
Webhooks
APIs
Endpoints
Requests
Responses
Errors
Exceptions
Middleware
Guards
Policies
Gates
Scopes
Observers
Events
Listeners
Jobs
Queues
Workers
Schedules
Commands
Tasks
Processes
Threads
Locks
Semaphores
Mutexes
Transactions
Savepoints
Rollbacks
Commits
Connections
Pools
Drivers
Adapters
Repositories
Services
Controllers
Routes
Middleware